

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342

Кринкин Н.К.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Реализовать метод Гаусса, протестировать его на матрицах различного размера, проверить точность и скорость работы алгоритма.

Теоретический минимум.

Метод Гаусса это классический алгоритм последовательного исключения неизвестных для решения систем линейных уравнений. Он приводит матрицу системы к треугольному виду с помощью элементарных преобразований, после чего переменные находятся обратной подстановкой. Метод универсален и подходит для систем с любым числом решений.

Существуют модификации алгоритма для повышения точности:

- 1) Метод частичного выбора - на каждом шаге выбирается максимальный элемент из столбца;
- 2) Метод полного выбора - на каждом шаге выбирается максимальный элемент из всех элементов по индексу больших текущего.

Выполнение работы.

В качестве модификации метода Гаусса был выбран метод частичного выбора, трудоемкость по операциям составляет $O(\frac{2m^3}{3})$ при достаточной для учебных задач точности. На каждом шаге метода частичного выбора выбирается максимальный элемент в столбце, после чего происходит перестановка строки текущего ведущего элемента с найденным.

При запуске написанной программы, создается и заполняется числами выбранного типа (int, float, double, long double) квадратная матрица A размером N, которое вводит пользователь. Сгенерированная матрица передается в функцию, которая генерирует вектор X, содержащий решение системы уравнений, перемножением матрицы A на вектор X получает вектор B и возвращает пару X, B. После, в зависимости от выбора пользователя, матрица A

и вектор V передается в функцию, которая возвращает вектор решения $\bar{X}$. Далее вектор X и $\bar{X}$ сравниваются.

Реализованный алгоритм был протестирован на точность и скорость для различных размеров матриц N (от 1 до 10000). Результаты погрешности и скорости выполнения представлены на Рисунке 1 и Рисунке 2.

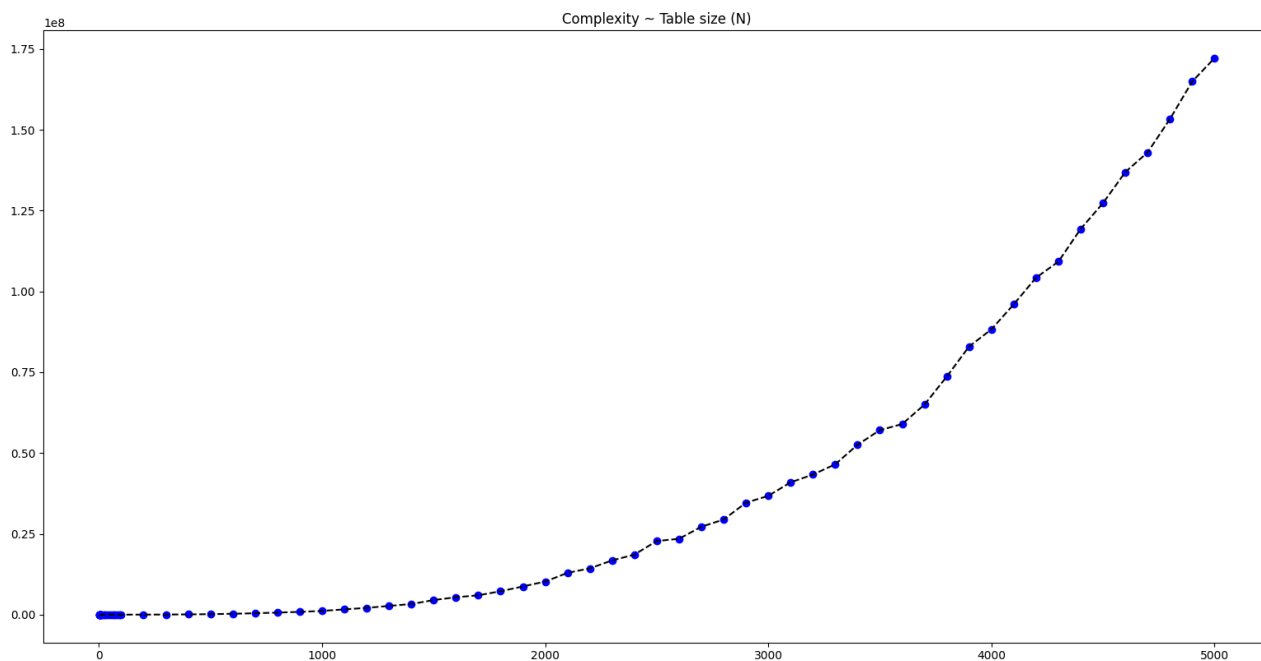


Рисунок 1 - График скорости алгоритма

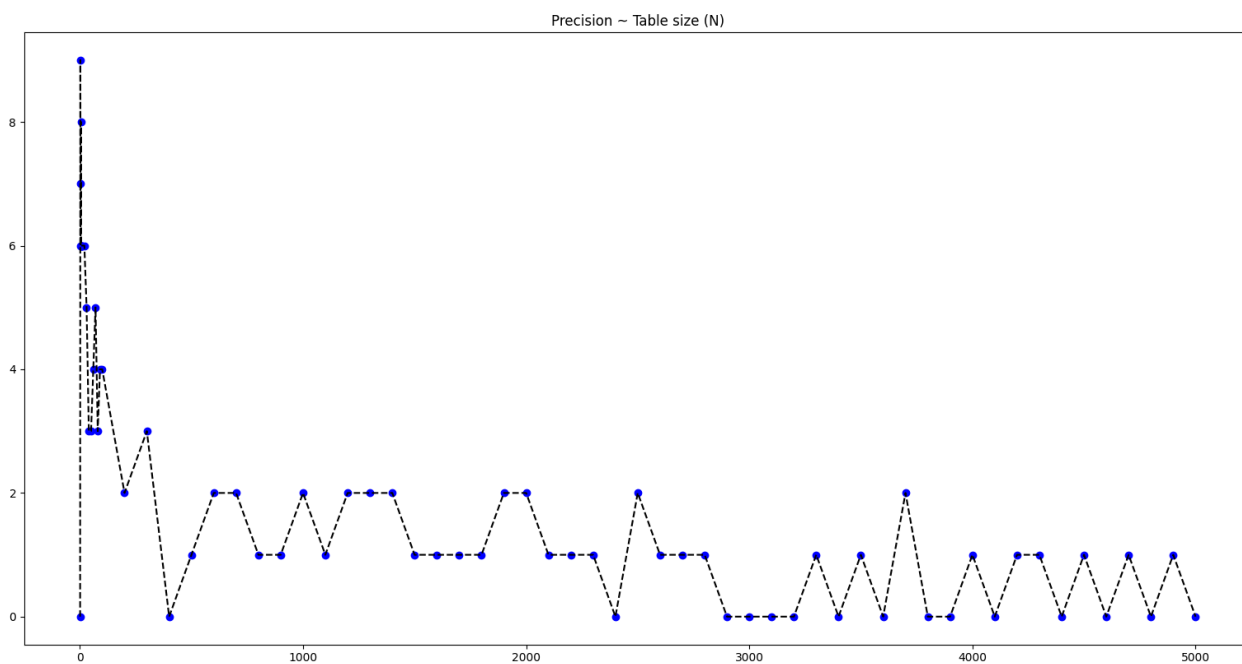


Рисунок 2 - Сравнение погрешности

Выводы.

В ходе выполнения работы был реализован метод Гаусса с частичным выбором, проведен анализ времени работы (быстродействия) и относительной погрешности реализованного метода.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл lab.c

```
#include <time.h>
#include <stdio.h>
#include <stdlib.h>

#define ABS(num) (num < 0 ? -num : num)
#define M_TYPE float

typedef struct {
    M_TYPE **data;
    int n;
} matrix_t;

void print_table(matrix_t *matt) {
    if (matt) {
        for (int i=0; i<matt->n; i++) {
            for (int j=0; j<matt->n; j++) {
                if (sizeof(M_TYPE) == sizeof(float)) printf("%8f ",
matt->data[i][j]);
                if (sizeof(M_TYPE) == sizeof(double)) printf("%8lf ",
matt->data[i][j]);
            }
            // print b[i]
            if (sizeof(M_TYPE) == sizeof(float)) printf("| %8f ",
matt->data[i][matt->n]);
            if (sizeof(M_TYPE) == sizeof(double)) printf("| %8lf ",
matt->data[i][matt->n]);
            printf("\n");
        }
    }
}

matrix_t *create_empty(int n) {
    matrix_t *mat = malloc(sizeof(matrix_t));
    if (mat) {
        mat->n = n;
        mat->data = calloc(n, sizeof(M_TYPE **));
        for (int i=0; i<n; i++) {
            ((M_TYPE **) mat->data)[i] = (M_TYPE *) calloc(n + 1, sizeof(M_TYPE));
        }
    }
    return mat;
}

matrix_t *generate_random(int n) {
    matrix_t *mat = create_empty(n);
    if (n) {
        for (int i=0; i<n; i++) {
            for (int j=0; j<n + 1; j++) {
                if (sizeof(M_TYPE) == sizeof(float)) {
                    mat->data[i][j] = (float) rand() / (float) rand();
                }
                if (sizeof(M_TYPE) == sizeof(double)) {
                    mat->data[i][j] = (float) rand() / (float) rand();
                }
            }
        }
    }
    return mat;
}
```

```

}

matrix_t *from_file(const char *path) {
    matrix_t *mat = 0;
    FILE *file = fopen(path, "r");
    if (file) {
        int n;
        fscanf(file, "%d\n", &n);
        if (n > 0) {
            mat = create_empty(n);
            if (mat) {
                for (int i=0; i<n; i++) {
                    for (int j=0; j<n + 1; j++) {
                        if (sizeof(M_TYPE) == sizeof(float)) {
                            fscanf(file, "%f", mat->data[i] + j);
                        }
                        if (sizeof(M_TYPE) == sizeof(double)) {
                            fscanf(file, "%lf", mat->data[i] + j);
                        }
                    }
                }
            }
        }
    } else {
        printf("File not found!\n");
    }
    if (file) fclose(file);

    return mat;
}

void delete_matrix(matrix_t * matrix) {
    if (matrix) {
        for (int i=0; i<matrix->n; i++)
            free(matrix->data[i]);
        free(matrix->data);
        free(matrix);
    }
}

void swap_rows(int i, int j, matrix_t *matrix) {
    M_TYPE *tmp = matrix->data[i];
    matrix->data[i] = matrix->data[j];
    matrix->data[j] = tmp;
}

int get_row_max_col(matrix_t *mat, int idx) {
    int max_idx = -1;
    M_TYPE max_num = 0;
    for (int it=idx; it<mat->n; it++) {
        if (ABS(mat->data[it][idx]) > ABS(max_num)) {
            max_num = mat->data[it][idx];
            max_idx = it;
        }
    }
    return max_idx;
}

// returns solutions
M_TYPE *Gauss_solve(matrix_t *matt) {
    int m_idx;
    int broke = 0;
    M_TYPE *sols;

```

```

if (matt) {
    sols = (M_TYPE *) calloc(matt->n, sizeof(M_TYPE));

    // direct move
    for (int it=0; it<matt->n; it++) {
        // swap row with max column element
        m_idx = get_row_max_col(matt, it);
        if (m_idx == -1) {
            printf("Infinite solutions case! Over\n");
            broke = 1;
            break;
        }
        if (m_idx != it) spaw_rows(it, m_idx, matt);
        // subtract from each
        for (int s_it = it + 1; s_it<matt->n; s_it++) {
            M_TYPE sub = matt->data[s_it][it] / matt->data[it][it];
            for (int k=it; k<matt->n + 1; k++) {
                matt->data[s_it][k] -= sub * matt->data[it][k];
            }
        }
    }

    // reverse move
    for (int i=matt->n - 1; (i>=0) && (! broke); i--) {
        M_TYPE sum = 0;
        for (int j=i + 1; j<matt->n; j++) {
            sum += matt->data[i][j] * sols[j];
        }
        sols[i] = (matt->data[i][matt->n] - sum) / matt->data[i][i];
    }
}

return sols;
}

int main(int argc, char *argv[]) {
    srand(time(NULL));

    int n;
    matrix_t *matrix = 0;
    if (argc == 1) {
        printf("Generating random table\n");
        printf("input table size: ");
        scanf("%d", &n);
        if (n <= 0) {
            printf("Incorrect size!\n");
            goto end;
        }
        matrix = generate_random(n);
    }

    if (argc == 2) {
        printf("Getting table from file\n");
        matrix = from_file(argv[1]);
    }

    if (matrix) {
        printf("Your table: \n");
        print_table(matrix);
    }
}

```



```

        if (matrix) {
            M_TYPE *sols = Gauss_solve(matrix);
            if (sols) {
                for (int i=0; i<matrix->n; i++) {
                    if (sizeof(M_TYPE) == sizeof(float)) printf("X%d = %f\n", i,
sols[i]);
                    if (sizeof(M_TYPE) == sizeof(double)) printf("X%d = %lf\n", i,
sols[i]);
                }
            }
        }
    }

end:
    if (matrix) delete_matrix(matrix);
    return 0;
}

```

Файл research.c

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>

#define ABS(num) (num < 0 ? -num : num)
#define M_TYPE float

typedef struct {
    M_TYPE **data;
    int n;
} matrix_t;

void print_table(matrix_t *matt) {
    if (matt) {
        for (int i=0; i<matt->n; i++) {
            for (int j=0; j<matt->n; j++) {
                if (sizeof(M_TYPE) == sizeof(float)) printf("%8f ",
matt->data[i][j]);
                if (sizeof(M_TYPE) == sizeof(double)) printf("%8lf ",
matt->data[i][j]);
            }
            // print b[i]
            if (sizeof(M_TYPE) == sizeof(float)) printf("| %8f ",
matt->data[i][matt->n]);

```

```

        if (sizeof(M_TYPE) == sizeof(double)) printf("| %8lf ",
matt->data[i][matt->n]);
        printf("\n");
    }
}

}

matrix_t *create_empty(int n) {
    matrix_t *mat = malloc(sizeof(matrix_t));
    if (mat) {
        mat->n = n;
        mat->data = calloc(n, sizeof(M_TYPE **));
        for (int i=0; i<n; i++) {
            ((M_TYPE **) mat->data)[i] = (M_TYPE *) calloc(n + 1, sizeof(M_TYPE));
        }
    }
    return mat;
}

matrix_t *generate_random_sols(int n, M_TYPE *sols) {
    matrix_t *mat = create_empty(n);
    if (n) {
        // fill mat[i][j]
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                if (sizeof(M_TYPE) == sizeof(float)) {
                    mat->data[i][j] = (float) rand() / (float) rand();
                }
                if (sizeof(M_TYPE) == sizeof(double)) {
                    mat->data[i][j] = (float) rand() / (float) rand();
                }
            }
        }
        // generate sols
        for (int i=0; i<n; i++) {
            if (sizeof(M_TYPE) == sizeof(float)) {
                sols[i] = (float) rand() / (float) rand();
            }
            if (sizeof(M_TYPE) == sizeof(double)) {
                sols[i] = (float) rand() / (float) rand();
            }
        }
    }
}

```

```

        // calculate b[i]
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                mat->data[i][n] += sols[j] * mat->data[i][j];

            }
        }
    }
    return mat;
}

matrix_t *generate_random(int n) {
    matrix_t *mat = create_empty(n);
    if (n) {
        for (int i=0; i<n; i++) {
            for (int j=0; j<n + 1; j++) {
                if (sizeof(M_TYPE) == sizeof(float)) {
                    mat->data[i][j] = (float) rand() / (float) rand();
                }
                if (sizeof(M_TYPE) == sizeof(double)) {
                    mat->data[i][j] = (float) rand() / (float) rand();
                }
            }
        }
    }
    return mat;
}

matrix_t *from_file(const char *path) {
    matrix_t *mat = 0;
    FILE *file = fopen(path, "r");
    if (file) {
        int n;
        fscanf(file, "%d\n", &n);
        if (n > 0) {
            mat = create_empty(n);
            if (mat) {
                for (int i=0; i<n; i++) {
                    for (int j=0; j<n + 1; j++) {
                        if (sizeof(M_TYPE) == sizeof(float)) {
                            fscanf(file, "%f", mat->data[i] + j);
                        }
                    }
                }
            }
        }
    }
}

```

```

        if (sizeof(M_TYPE) == sizeof(double)) {
            fscanf(file, "%lf", mat->data[i] + j);
        }
    }
}

}

}

} else {
    printf("File not found!\n");
}

if (file) fclose(file);

return mat;
}

void delete_matrix(matrix_t * matrix) {
    if (matrix) {
        for (int i=0; i<matrix->n; i++)
            free(matrix->data[i]);
        free(matrix->data);
        free(matrix);
    }
}

void swap_rows(int i, int j, matrix_t *matrix) {
    M_TYPE *tmp = matrix->data[i];
    matrix->data[i] = matrix->data[j];
    matrix->data[j] = tmp;
}

int get_row_max_col(matrix_t *mat, int idx) {
    int max_idx = -1;
    M_TYPE max_num = 0;
    for (int it=idx; it<mat->n; it++) {
        if (ABS(mat->data[it][idx]) > ABS(max_num)) {
            max_num = mat->data[it][idx];
            max_idx = it;
        }
    }
    return max_idx;
}

```

```

// returns presicion in form of decimal places
int calculate_diff(int n, M_TYPE *orig_sols, M_TYPE *sols) {
    M_TYPE diff;
    M_TYPE max_diff = 0;
    for (int i=0; i<n; i++) {
        diff = ABS(sols[i] - orig_sols[i]);
        max_diff = diff > max_diff ? diff : max_diff;
    }
    int p = 0;
    while (max_diff < 1 && max_diff) {
        p++;
        max_diff *= 10;
    }
    return p;
}

// returns solutions
M_TYPE *Gauss_solve(matrix_t *matt) {
    int m_idx;
    int broke = 0;
    M_TYPE *sols;

    if (matt) {
        sols = (M_TYPE *) calloc(matt->n, sizeof(M_TYPE));

        // direct move
        for (int it=0; it<matt->n; it++) {
            // swap row with max column element
            m_idx = get_row_max_col(matt, it);
            if (m_idx == -1) {
                printf("Infinite solutions case! Over\n");
                broke = 1;
                break;
            }
            if (m_idx != it) swap_rows(it, m_idx, matt);
            // subtract from each
            for (int s_it = it + 1; s_it<matt->n; s_it++) {
                M_TYPE sub = matt->data[s_it][it] / matt->data[it][it];
                for (int k=it; k<matt->n + 1; k++) {
                    matt->data[s_it][k] -= sub * matt->data[it][k];
                }
            }
        }
    }
}

```

```

    }

    // reverse move
    for (int i=matt->n - 1; (i>=0) && (! broke); i--) {
        M_TYPE sum = 0;
        for (int j=i + 1; j<matt->n; j++) {
            sum += matt->data[i][j] * sols[j];
        }
        sols[i] = (matt->data[i][matt->n] - sum) / matt->data[i][i];
    }
}

return sols;
}

void step(int n, FILE* file) {
    clock_t start, end;
    double dur;
    M_TYPE *orig = (M_TYPE *) calloc(n, sizeof(M_TYPE));
    matrix_t *matrix = generate_random_sols(n, orig);
    start = clock();
    M_TYPE *sols = Gauss_solve(matrix);
    // save time in ms
    end = clock();
    dur = (end - start) * 1000 * 1000 / CLOCKS_PER_SEC;
    float pr = calculate_diff(n, orig, sols);
    fprintf(file, "%d %lf %f\n", n, dur, pr);
    free(orig);
    free(sols);
    delete_matrix(matrix);
}

int main(int argc, char * argv[]) {
    srand(time(NULL));
    matrix_t *matrix = 0;
    // explore complexity and precision
    // from 1 to 9
    FILE *file = fopen(argv[1], "w");
    for (int n=1; n<10; n++) {
        step(n, file);
    }
    // from 10 to 100 with step 10

```

```

    for (int n=10; n<100; n += 10) {
        step(n, file);
    }
    // from 100 to 10'000 with step 100
    for (int n=100; n<=5000; n += 100) {
        step(n, file);
    }

    return 0;
}

```

Файл graph.py

```

import matplotlib
matplotlib.use("TkAgg")
from matplotlib import pyplot as plt
import sys

data = open(sys.argv[1], "r").readlines()
xs = []
comps = []
precs = []

for line in data:
    if line:
        x, y1, y2 = list(map(float, line.split()))
        xs.append(x)
        comps.append(y1)
        precs.append(y2)

plt.plot(xs, comps, "bo", label="Time (ms)")
plt.plot(xs, comps, "k--")
plt.title("Complexity ~ Table size (N)")
plt.show()

plt.plot(xs, precs, "bo", label="Precision in decimal places")
plt.plot(xs, precs, "k--")
plt.title("Precision ~ Table size (N)")
plt.show()

```

Файл Makefile

```

SRC_DIR = ${CURDIR}/src
OBJ_DIR = ${CURDIR}/objs

```

```

EXP_DIR = ${CURDIR}/examples

FN          = ${OBJ_DIR}/check.txt

lab: ${OBJ_DIR}/lab.o
    ${OBJ_DIR}/lab.o

research: ${SRC_DIR}/graph.py
    python3 ${SRC_DIR}/graph.py ${FN}

${FN}: ${OBJ_DIR}/research.o
    ${OBJ_DIR}/research.o ${FN}

${OBJ_DIR}:
    mkdir ${OBJ_DIR}

${OBJ_DIR}/lab.o: ${OBJ_DIR} ${SRC_DIR}/lab.c
    @gcc -g3 ${SRC_DIR}/lab.c -o ${OBJ_DIR}/lab.o -Wformat=0

${OBJ_DIR}/research.o: ${OBJ_DIR} ${SRC_DIR}/research.c
    @gcc -g3 ${SRC_DIR}/research.c -o ${OBJ_DIR}/research.o -Wformat=0

clear:
    rm ${OBJ_DIR}/*

help:
    @echo "Available options:"
    @echo "    make [lab] -- solve matrix via Gauss"
    @echo "    make speed-comp -- compare speed with np.linalg"
    @echo "    make input_errors -- compare precision with np.linalg"

```